Informe de QA — Módulo de Pedidos

Laboratorios ABD · Qué tenía el sistema, qué se mejoró y qué sigue mal

Sistema original: ABDFacturacion (Java EE / JSF) · **Nuevo:** pedidos-react · **Base:** Oracle 12c · PDBABD (producción)
Fecha: 4 de agosto de 2026 · **Repositorio:** github.com/edgar2420/FacturacionABD · **Rama:** main

Cómo leer este informe. Todo lo que aquí se afirma está verificado contra la base de datos real o contra el código fuente, y se indica de dónde sale cada dato. Cuando algo no se ha podido comprobar, se dice expresamente. Ninguna comprobación modificó datos de producción.

1. Resumen

Área	Antes	Ahora
Filas por producto en la búsqueda	1 por lote	1 por producto
Saldo que ve el vendedor	el de un lote suelto	suma de todos
Elección de lote al vender	manual	automática (FEFO)
Paginación de búsquedas	ninguna	15 por página
Clientes devueltos en una búsqueda	hasta 7.108	15
Pruebas automáticas	0	124
Consultas con inyección SQL	sí	no
Uso en celular	tabla cortada	tarjetas

Defectos encontrados: 11. De ellos **5 corregidos** en la nueva aplicación, **4 siguen abiertos** porque están en la base de datos o en el sistema JSF, y **2 son latentes** (no hacen daño hoy, pero lo harán si cambian los datos).

2. Qué tenía la aplicación antes

2.1 Tecnología

Lenguaje y plataforma	Java 8, Java EE 7 Web
Interfaz	JSF 2.2 (Facelets) con PrimeFaces 6.0, renderkit PRIMEFACES_MOBILE
Persistencia	Hibernate 4.3.10 con mapeos XML (`.hbm.xml`)
Base de datos	Oracle 12c — esquemas ABD, IBSVENTA, IBSADM, IBSPLA
Informes	JasperReports 5.6, iText, PDFBox, ZXing (QR)
Servidor	Payara 5 / GlassFish
Compilación	Apache Ant + NetBeans. Sin Maven: los JAR van por ruta absoluta

2.2 Dónde está la lógica de negocio

No está en Java. Vive en el paquete `ABD.PCK_OPERACIONES` de Oracle. Las clases Java solo orquestan: abren la conexión, llaman a la función y traducen el resultado.

Función del paquete	Qué hace
`F_VALIDA_USUARIO`	Comprueba usuario y contraseña
`F_OBTENER_PRODUCTOS`	Catálogo con precio y saldo, **una fila por lote**
`F_CREAR_PEDIDO`	Crea la cabecera y la primera línea a la vez
`F_CREAR_DETALLE_PEDIDO`	Añade una línea más
`F_ELIMINAR_DETALLE_PEDIDO`	Borra líneas **por código de ítem**
`F_TERMINAR_PEDIDO`	Totaliza y pasa el estado a «V»
`F_PAGAR_PEDIDO`	Registra forma de pago y datos de factura

Consecuencia práctica: reescribir la interfaz no cambia las reglas del negocio. La aplicación nueva llama a las mismas funciones.

2.3 Organización por almacenes

Todo el sistema gira alrededor del almacén, que sale de la columna `UNI_NEG` de `ABD.USUARIOS`. Hay ocho:

Almacén	Usuarios	Productos con saldo	Corresponde a
`AL-VEN-ABD-FAB`	87	110	Santa Cruz (90 % de su histórico)
`AL-SUM-ABD-FAB`	31	0	Suministros de planta, no vende
`AL-VEN-ABD-LPZ`	29	96	La Paz
`AL-VEN-ABD-CB1`	27	111	Cochabamba (80 % de su histórico)
`AL-VEN-ABD-SUC`	8	93	Sucre (95 % de su histórico)
`AL-VEN-ABD-SC8`	3	0	Santa Cruz, sin stock cargado
`AL-VEN-ABD-ORU`	3	83	Oruro
`AL-VEN-ABD-ARG`	1	0	Argentina

BIEN **El aislamiento por almacén funciona y siempre funcionó.** De 16.547 pedidos históricos, **16.544** tienen el almacén que corresponde al `UNI_NEG` de quien los grabó. Ninguna persona tiene cuenta en dos almacenes y ningún usuario grabó nunca en dos. Los 3 restantes son de usuarios ya dados de baja.

2.4 El problema de los lotes

El catálogo devolvía **una fila por cada lote**. Para el almacén de Fábrica: **233 filas** para solo **110 productos**; 60 de esos productos tienen más de un lote.

El vendedor tenía que elegir el lote a mano y solo veía el saldo de ese lote. Ejemplo real del producto PT982:

Lote	Vence	Saldo
`F244`	30/09/26	5
`F245`	30/09/26	120

Lote	Vence	Saldo
F250	31/12/26	200
Disponible de verdad		**325**

Si el cliente pedía 300, el vendedor veía tres filas y ninguna llegaba a 300. **Parecía que no había stock, habiendo de sobra.**

3. Defectos encontrados

3.1 Graves

GRAVE ABIERTO **D-01 · La contraseña no se comprueba si el usuario no tiene clave guardada**

Dónde: `ABD.PCK_OPERACIONES.F_VALIDA_USUARIO`

Cuando el usuario no tiene fila en la tabla de claves, la variable interna queda a NULL. La comparación `x_encriptado != v_clave` contra NULL no da ni verdadero ni falso: da **desconocido**. El `IF` nunca se cumple y la función devuelve `'1'`, es decir «acceso concedido», **con cualquier contraseña**.

Afecta también al sistema JSF en producción, no solo al nuevo.

No corregido: está en la base de datos, y la instrucción fue no tocarla. Requiere revisión por parte de quien la administra.

GRAVE CORREGIDO **D-02 · Inyección SQL en la búsqueda de clientes**

Dónde: `PedidosDao.buscarCliente()`, línea 125

```
sql = "from Personas where upper(nombre) like '%"+valor.toUpperCase()+"%' and codigoCliente='SI'
order by nombre";
```

El texto que escribe el usuario se pega dentro de la consulta. Una comilla simple basta para alterarla. El patrón se repite en varias consultas del mismo archivo.

Corregido en la aplicación nueva: se usan variables de vínculo (*bind variables*), el valor viaja aparte de la consulta. *El código Java original no se modificó.*

GRAVE ABIERTO **D-03 · El precio que se muestra no es el que se graba**

Dónde: `F_OBTENER_PRODUCTOS` frente a `F_CREAR_DETALLE_PEDIDO`

La pantalla pide el precio con la **fecha de entrega**:

```
IBSADM.get_precio('P', i.codigo, p_cliente, p_almacen, to_date(p_fecha,'dd/mm/yyyy'))
```

Pero al grabar la línea se calcula con **la fecha de hoy**:

```
v_fecha   DATE := SYSDATE;
v_precio := IBSADM.GET_IMPORTE('P', p_item, p_cliente, p_almacen, v_fecha, 'BS', p_cantidad, v_dcto);
```

Comprobado en la base con el producto PT982 y el cliente I00060:

Cálculo	Fecha usada	Precio
Lo que ve el vendedor	04/09/2026 (entrega)	Bs 50,00
Lo que se graba	hoy (SYSDATE)	**Bs 45,00**

Una diferencia del **10 %** en este producto. Ocurre siempre que la fecha de entrega caiga en un tramo de precio distinto al de hoy.

No corregido: el cálculo está dentro del PL/SQL. La aplicación nueva llama al mismo procedimiento y por tanto **hereda el defecto**. Corregirlo exige cambiar el paquete.

3.2 Medios

MEDIO CORREGIDO **D-04 · Búsquedas sin límite**

Ninguna consulta tenía paginación. Buscar clientes por la letra «A» devolvía **7.108 registros** en una sola respuesta, todos a la pantalla.

Corregido: 15 por página, tope de 50. Los productos, que vienen de un cursor, se agrupan antes de paginar para que los lotes de un mismo producto no queden partidos entre dos páginas.

MEDIO CORREGIDO **D-05 · Llamada mal escrita al eliminar una línea**

Dónde: `PedidosDao.eliminarDetallePedido()`, línea 671

```
conn.prepareCall("{? = call ABD.PCK_OPERACIONES.F_ELIMINAR_DETALLE_PEDIDO(?,?,?,?)");
```

Falta la llave de cierre `}`. La cadena está mal formada. Además el mensaje de error del `catch` dice `F_ELIMINARAR_DETALLE_PEDIDO`, con una sílaba de más.

Corregido en la aplicación nueva: la llamada se hace correctamente.

MEDIO ABIERTO **D-06 · Credenciales en texto plano dentro del repositorio**

`src/java/hibernate.cfg.xml` y los `constantes*.properties` llevan usuario y contraseña de Oracle escritos tal cual. `web/WEB-INF/web.xml` lleva las claves de reCAPTCHA.

No corregido por decisión de no modificar el proyecto Java. Si el repositorio se hace público o se comparte, esas credenciales deben rotarse **antes**. En la aplicación nueva las credenciales están en un archivo `.env` excluido del control de versiones.

MEDIO ABIERTO **D-07 · No existe relación entre cliente y almacén**

`ABD.PERSONAS` solo tiene una columna `CIUDAD` de texto libre. Nada impide que un vendedor de un departamento le venda a un cliente de otro. El sistema JSF tampoco lo impide: su consulta no filtra por almacén ni por ciudad.

Y los datos no permiten deducirlo con seguridad. En el histórico conviven:

```
SANTA CRUZ · SANTA RUZ · SAANTA CRUZ · SANTA CRU · SABTA CRUZ
SNTA CRUZ · SANTA CRUZZ · SANTAC RUZ · SANT CRUZ · SANTA CRUZ-MONTERO
SANTA CRUZ-WARNES · SATELITE · EL ALTO · LA PAZ - EL ALTO
```

Mitigado, no resuelto: la aplicación nueva **avisa** cuando la ciudad del cliente no corresponde al almacén, pero **no bloquea**. Bloquear con estos datos dejaría fuera clientes legítimos. La solución de fondo es una columna de almacén en `PERSONAS`, y eso toca la base.

3.3 Latentes — no molestan hoy, molestarán mañana

MEDIO LATENTE **D-08 · El almacén se compara con LIKE, no por igualdad**

Dónde: `F_OBTENER_PRODUCTOS`

```
and s.codigo_almacen LIKE '%'||p_almacen||'%'
```

Pedir el almacén `AL-VEN-ABD-CB1` devuelve también las filas de otros dos que lo contienen como subcadena:

Almacén que casa	Filas	Saldo
AL-VEN-ABD-CB1	3.087	171.756
.AL-VEN-ABD-CB1	8	0
3AL-VEN-ABD-CB1	2	0

Hoy no causa error porque esos almacenes tienen saldo cero y el filtro los descarta. Pero el día que alguien cargue existencias en `.AL-VEN-ABD-CB1`, se sumarán al catálogo de Cochabamba **sin que nadie lo note**. Lo mismo ocurre con FAB, LPZ y ARG.

No corregido: está dentro del PL/SQL.

MEDIO LATENTE **D-09 · El saldo mostrado no descuenta lo pendiente de entrega**

La misma consulta filtra por `HAVING SUM(saldo_actual - pendiente_entrega) > 0` pero **devuelve** `SUM(saldo_actual)`, sin restar. El número que se enseña es mayor que lo realmente disponible.

Hoy no causa error: se revisaron las 14.749 filas de stock del almacén de Fábrica y **ninguna** tiene `pendiente_entrega` distinto de cero. La columna no se está usando.

Si algún día se empieza a usar, el vendedor verá más stock del que hay y se podrá vender por encima del disponible.

3.4 Leves

LEVE CORREGIDO **D-10 · Etiquetas de formulario sin asociar a su campo**

En la aplicación nueva, el componente `Campo` pintaba `<label>` e `<input>` sueltos. Un lector de pantalla leía la caja sin decir para qué era, y tocar la etiqueta en el celular no daba el foco al campo. Detectado al escribir las pruebas.

Corregido con `id` / `htmlFor`.

LEVE ABIERTO **D-11 · Eliminar una línea borra todas las del mismo producto**

`F_ELIMINAR_DETALLE_PEDIDO` borra con `WHERE codigo_item = ...`, sin el número de línea. Si un producto ocupa varias líneas, se van todas.

No es un problema con el reparto por lotes —al contrario, es justo lo que se quiere: el vendedor quita «el producto», no «un lote»— pero conviene saberlo, porque no hay forma de borrar una sola línea.

4. Qué se mejoró

4.1 Lotes sumados y reparto automático

Es el cambio de fondo. Se pidió expresamente y se implementó así:

ANTES
Tres filas de PT982, saldos 5, 120 y 200.
El vendedor elige el lote a mano.
Pidiendo 300 parece que no hay stock.

AHORA
Una fila de PT982, saldo 325, marca «3 lotes».
El vendedor solo escribe la cantidad.
El sistema reparte solo.

La regla de despacho (FEFO — sale antes lo que caduca antes):

1. Primero el lote que **vence antes**.
2. Si dos vencen el mismo día, primero el de **menor saldo**, para liquidar los restos.
3. Un lote sin fecha legible se deja para el final: no se puede afirmar que venza pronto y sacarlo primero sería arriesgado.

Pidiendo 300 unidades de PT982:

Lote	Vence	Saldo	Sale	Motivo
F244	30/09/26	5	**5**	vence antes; de los dos del 30/09 es el más pequeño
F245	30/09/26	120	**120**	mismo vencimiento, más saldo
F250	31/12/26	200	**175**	vence después, se usa al final

Detalles de implementación que importan:

- **El servidor relee el producto al confirmar.** No se fía de lo que manda el navegador: entre que el vendedor busca y confirma pueden pasar minutos y otro puede haberse llevado el lote.
- **Cada lote es una línea del pedido**, porque cada fila de `LIN_PEDIDOS` guarda un solo lote. Pedir 300 genera tres líneas.
- **Si el saldo total no alcanza, no se reparte nada** y se avisa cuánto falta. Es preferible a dejar el pedido a medias.
- **El vendedor ve el reparto antes de confirmar**, mientras escribe la cantidad. No es una caja negra.

Limitación conocida. Los procedimientos de Oracle hacen `COMMIT` dentro de sí mismos, así que un reparto en varios lotes **no puede envolverse en una transacción**. Si un tramo falla a mitad se compensa borrando las líneas del producto —y solo si el producto no estaba ya en el pedido, porque si estaba se llevaría líneas buenas—. Cuando no se puede compensar, se informa el estado real y decide la persona.

4.2 Interfaz

Aspecto	Antes (JSF)	Ahora (React)
Elegir cliente o producto	seleccionar arriba, bajar y pulsar un botón al pie	se toca la fila y se abre su ficha, se confirma ahí mismo
Tablas en celular	se cortaban, con desplazamiento horizontal	se convierten en tarjetas con etiqueta y valor

Aspecto	Antes (JSF)	Ahora (React)
Fecha de entrega	se escribía a mano	calendario del propio teléfono, con hoy como mínimo, más atajos «Hoy» y «Mañana»
Avisos	mensajes de JSF fáciles de pasar por alto	los errores en modal centrado; los éxitos en aviso que se va solo
Tema	único	arranca siempre en claro; el oscuro se recuerda si se elige
Almacén de trabajo	código en letra pequeña, solo en pantalla grande	nombre legible siempre visible, también en celular
Iconos	Font Awesome	SVG propios, sin librería externa ni emojis
Instalación	solo navegador	instalable como aplicación (PWA), abre sin conexión

Nota sobre la PWA: los navegadores solo permiten instalar por HTTPS o desde `localhost`. Para instalarla en los celulares de los vendedores hará falta servirla con certificado.

4.3 Seguridad y solidez

- **Variables de vínculo** en todas las consultas: se cierra la inyección SQL.
- **El almacén nunca llega del navegador.** Sale de `UNI_NEG` al iniciar sesión y vive en la sesión del servidor. Manipular la petición no sirve para pedir stock de otro almacén.
- **Las credenciales salen del código** y pasan a un `.env` excluido del repositorio.
- **Validación también en el servidor.** Lo que valida el navegador es comodidad; la validación que manda es la del servidor.
- **Los errores de Oracle no llegan al navegador:** se registran en el servidor y al usuario le llega un mensaje neutro.

4.4 Pruebas automáticas

El sistema original no tenía ninguna. Ahora hay **124**.

Suite	Pruebas	Qué cubre	Cómo se lanza
Backend — reglas y lotes	51	reparto FEFO, fechas, cantidades, regiones, bancos	`npm test`
Frontend — Cypress	63	sesión, cliente, vendedor, lotes, pedido, pago, interfaz	`npm run qa`
Integración con Oracle	10	el reparto contra la base real, en tablas de prueba	`npm run prueba:reparto`
Total	**124**	todas en verde	

Las pruebas de Cypress no tocan la base. Todas las llamadas a la API se responden con datos de prueba. No es comodidad: el backend apunta a producción y una prueba que creara un pedido crearía un pedido real.

La prueba de integración usa tablas aparte. Escribe en `IBSVENTA.PPEDIDOS` y `PLIN_PEDIDOS`, copias exactas de las reales, y se limpia sola al terminar. Verificado después de ejecutarla: las tablas de prueba quedaron en cero y ninguno de los pedidos reales del día tenía el usuario de pruebas.

5. Qué falta y qué se recomienda

5.1 No probado

El paquete `ABD.PCK_OPERACIONES` no se ha ejercitado en la prueba de integración: sus `INSERT` están dentro del PL/SQL y apuntan a las tablas reales, así que no se pueden redirigir desde el servidor. Lo que sí quedó probado de punta a punta es la orquestación nueva —el reparto en lotes—, que es lo que no existía antes. El paquete lleva años en producción.

Para cubrirlo haría falta una copia del paquete que escriba en las tablas de prueba, como ya existe `PCK_OPERACIONES_BACK`. Eso lo debe hacer quien administre la base.

5.2 Recomendaciones, por orden de urgencia

#	Defecto	Acción	Quién
1	D-01	Revisar `F_VALIDA_USUARIO`: usar `NVL` o `IS NULL` para que una clave ausente deniegue el acceso en vez de concederlo	Administrador de la base
2	D-03	Unificar la fecha de cálculo del precio: `F_CREAR_DETALLE_PEDIDO` debería recibir la fecha de entrega, no usar `SYSDATE`	Administrador de la base
3	D-06	Rotar las credenciales de Oracle y las claves de reCAPTCHA si el repositorio se comparte	Sistemas
4	D-08	Cambiar `LIKE` por `=` en el filtro de almacén, o limpiar los almacenes con punto y número delante	Administrador de la base
5	D-07	Valorar una columna de almacén en `PERSONAS` si se quiere bloquear de verdad la venta entre departamentos	Decisión de negocio
6	D-09	Si se empieza a usar `pendiente_entrega`, restarlo también en el saldo que se muestra	Administrador de la base
7	—	Revisar `server/src/zonas.js`: los almacenes LPZ, ORU, SC8 y ARG no tienen histórico y su región se dedujo del sufijo	Negocio
8	—	Servir la aplicación por HTTPS para poder instalarla en los celulares	Sistemas

5.3 Advertencias operativas

El entorno apunta a producción. El archivo `.env` tiene `192.168.0.5/PDBABD`. Cualquier pedido creado desde la aplicación nueva es un pedido real, visible en el sistema JSF.

Para probar sin riesgo, poner `MODO_PRUEBA=1`: los pedidos se graban en las tablas de prueba. Los catálogos y saldos se siguen leyendo de producción, que es lo que interesa. **Dejar vacío en cualquier despliegue real.**

La variable `ALMACEN_DEV` es solo para desarrollo: permite entrar a cuentas que no están en `ABD.USUARIOS` asignándoles un almacén fijo. También debe quedar vacía en producción.

6. Cómo comprobar todo esto

```
cd pedidos-react/server && npm test             # 51 pruebas, sin base de datos
cd pedidos-react/web    && npm run dev          # levantar la interfaz
cd pedidos-react/qa     && npm run qa           # 63 pruebas de interfaz
```

```
# Con MODO_PRUEBA=1 en el .env:
cd pedidos-react/server && npm run prueba:reparto # 10 comprobaciones contra Oracle
```

Si Cypress no arranca y dice bad `option: --smoke-test`, la causa es la variable de entorno `ELECTRON_RUN_AS_NODE=1`, que hace que arranque como Node. Reinstalar el binario no sirve de nada.

Informe generado a partir de verificaciones sobre la base de datos Oracle de producción (solo lecturas) y sobre el código fuente de ambos sistemas. Los datos de saldos, lotes, precios y pedidos citados son reales y estaban vigentes el 4 de agosto de 2026.